МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №3
з дисципліни "Функціональне програмування"
на тему "НЕЗДАТНІ ДО ЗМІН СТРУКТУРИ ДАНИХ (IMMUTABILITY)"
Варіант 12. Ресторанне меню

Виконав:
Студент групи ФЕП-23с
Малецький Маркіян Володимирович

Перевірив:
проф. Свелеба С. А.

Львів – 2026

Мета: зрозуміти та застосувати незмінні структури даних Python (tuple, frozenset, @dataclass(frozen=True, slots=True), MappingProxyType), замінити мутації предметної моделі ресторанного меню на повернення нових копій (pure updates), виміряти ефект переходу на тестах та перевірити хешованість і кешування.

Теоретичні відомості: Функціональний стиль спирається на чисті функції та незмінні структури. Незмінність дає референтну прозорість (вираз = його значення), просту композицію, безпечне спільне читання і хешованість: незмінний об'єкт може бути ключем словника, елементом множини та аргументом @lru_cache. У стандартній бібліотеці Python для цього є tuple замість list, frozenset замість set, @dataclass(frozen=True, slots=True) для доменних сутностей та MappingProxyType для read-only представлення словника. MappingProxyType захищає лише верхній рівень і показує зміни вихідного словника, тому перед створенням проксі джерело копіюють; для вкладених структур реалізують глибоке заморожування (freeze), яке рекурсивно перетворює dict/set/list на MappingProxyType/frozenset/tuple.

Хід роботи:

1. За варіантом 12 (ресторанне меню) зафіксував мутабельну модель «до» у mutable_model.py: страва як dict зі списком інгредієнтів, множиною алергенів і метаданими:

```
# mutable_model.py
from typing import Any, Dict, List, Set

def make_dish_dict(id: int, name: str, price: float) -> Dict[str, Any]:
    return {
        "id": id,
        "name": name,
        "price": price,
        "ingredients": [],
        "allergens": set(),
        "meta": {},
    }

def add_ingredient_inplace(dish: Dict[str, Any], name: str, cost: float, weight: float) -> None:
    dish["ingredients"].append({"name": name, "cost": cost, "weight": weight})

def add_allergen_inplace(dish: Dict[str, Any], allergen: str) -> None:
    dish["allergens"].add(allergen.lower())

def calculate_margin_dict(price: float, cost: float) -> float:
    if price <= 0:
        return 0.0
    return round(((price - cost) / price) * 100, 2)
```

2. У model.py описав незмінну модель: сутності Ingredient та Dish визначено через @dataclass(frozen=True, slots=True), інгредієнти у tuple, алергени у frozenset, метадані — MappingProxyType:

```
# model.py
from dataclasses import dataclass, field
from types import MappingProxyType
from typing import Mapping, Tuple

def make_meta(**kwargs: str) -> MappingProxyType[str, str]:
    return MappingProxyType(dict(kwargs))

@dataclass(frozen=True, slots=True)
class Ingredient:
    name: str
    cost: float
    weight_grams: float

    def __post_init__(self) -> None:
        if self.cost < 0:
            raise ValueError("Cost cannot be negative")
        if self.weight_grams <= 0:
            raise ValueError("Weight must be strictly positive")

@dataclass(frozen=True, slots=True)
class Dish:
    id: int
    name: str
    price: float
    ingredients: Tuple[Ingredient, ...] = ()
    allergens: frozenset[str] = frozenset()
    meta: Mapping[str, str] = field(default_factory=lambda: make_meta(), hash=False)

    def total_cost(self) -> float:
        return sum(i.cost for i in self.ingredients)

    def total_weight(self) -> float:
        return sum(i.weight_grams for i in self.ingredients)

    def has_allergen(self, allergen: str) -> bool:
        return allergen.strip().lower() in {a.lower() for a in self.allergens}
```

3. У core.py реалізував pure updates (add_ingredient, add_allergen, update_price, with_meta), кешований розрахунок calculate_margin_cached, функцію index_by_dish та глибоке заморожування freeze:

```
# core.py
from collections.abc import Mapping
from dataclasses import replace
from functools import lru_cache
from types import MappingProxyType
from typing import Any, Mapping as TMapping
from model import Dish, Ingredient

def add_ingredient(dish: Dish, name: str, cost: float, weight: float) -> Dish:
    return replace(dish, ingredients=dish.ingredients + (Ingredient(name, cost, weight),))

def add_allergen(dish: Dish, allergen: str) -> Dish:
    return replace(dish, allergens=dish.allergens | frozenset({allergen.strip().lower()}))

def update_price(dish: Dish, new_price: float) -> Dish:
    return replace(dish, price=new_price)

def apply_discount(dish: Dish, percent: float) -> Dish:
    return replace(dish, price=round(dish.price * (1.0 - percent / 100.0), 2))

def with_meta(dish: Dish, **updates: str) -> Dish:
    merged = dict(dish.meta)
    merged.update(updates)
    return replace(dish, meta=MappingProxyType(merged))

@lru_cache(maxsize=1024)
def calculate_margin_cached(price: float, cost: float) -> float:
    if price <= 0:
        return 0.0
    return round(((price - cost) / price) * 100.0, 2)

def index_by_dish(dishes: tuple[Dish, ...]) -> TMapping[Dish, float]:
    return MappingProxyType({d: calculate_margin_cached(d.price, d.total_cost()) for d in dishes})

def freeze(obj: Any) -> Any:
    if isinstance(obj, MappingProxyType):
        return obj
    if isinstance(obj, Mapping):
        return MappingProxyType({k: freeze(v) for k, v in obj.items()})
    if isinstance(obj, (set, frozenset)):
        return frozenset(freeze(x) for x in obj)
    if isinstance(obj, (list, tuple)):
        return tuple(freeze(x) for x in obj)
    return obj
```

4. Скрипт main.py демонструє ланцюжок із трьох pure-оновлень, незмінність оригіналу та роботу з кешем:

```
# main.py
from model import Dish, Ingredient, make_meta
from core import add_ingredient, add_allergen, with_meta, calculate_margin_cached, index_by_dish

def main() -> None:
    d0 = Dish(1, "Pasta Carbonara", 220.0, (Ingredient("Pasta", 30.0, 150.0),), frozenset({"gluten"}), make_meta
(cuisine="Italian"))
    d1 = add_ingredient(d0, "Guanciale", 60.0, 70.0)
    d2 = add_allergen(d1, "lactose")
    d3 = with_meta(d2, chef="Mario")

    print(f"d0 is d3: {d0 is d3} | d0 unchanged: {len(d0.ingredients) == 1}")
    print(f"Margin: {calculate_margin_cached(d3.price, d3.total_cost())}%")
    print("Cache info:", calculate_margin_cached.cache_info())
    idx = index_by_dish((d0, d3))
    print(f"Indexed {len(idx)} dishes successfully as dict keys.")

if __name__ == "__main__":
    main()
```

5. У tests/test_immutability.py реалізував тести на відсутність мутації, FrozenInstanceError при записі в поля та read-only статус метаданих:

```
# tests/test_immutability.py
import pytest
from dataclasses import FrozenInstanceError
from types import MappingProxyType
from model import Dish, Ingredient, make_meta
from core import add_ingredient, add_allergen, update_price, with_meta, freeze, calculate_margin_cached

@pytest.fixture
def sample_dish() -> Dish:
    return Dish(12, "Pizza Margherita", 180.0, (Ingredient("Dough", 20.0, 200.0),), frozenset({"gluten"}), make_
meta(category="Pizza"))

def test_add_ingredient_returns_new_dish(sample_dish: Dish) -> None:
    original = sample_dish
    updated = add_ingredient(sample_dish, "Cheese", 40.0, 100.0)
    assert updated is not original
    assert len(sample_dish.ingredients) == 1
    assert len(updated.ingredients) == 2

def test_frozen_fields_raise_error(sample_dish: Dish) -> None:
    with pytest.raises(FrozenInstanceError):
        sample_dish.price = 300.0

def test_meta_is_read_only(sample_dish: Dish) -> None:
    with pytest.raises(TypeError):
        sample_dish.meta["category"] = "FastFood"

def test_hashability_and_cache(sample_dish: Dish) -> None:
    calculate_margin_cached.cache_clear()
    calculate_margin_cached(200.0, 50.0)
    calculate_margin_cached(200.0, 50.0)
    assert calculate_margin_cached.cache_info().hits == 1
    assert len({sample_dish, sample_dish}) == 1
```

6. У tests/test_performance.py реалізував порівняння часу виконання на мутабельній моделі «до» та незмінній «після» (по 2000 операцій):

```
# tests/test_performance.py
from mutable_model import make_dish_dict, add_ingredient_inplace, calculate_margin_dict
from model import Dish, Ingredient
from core import add_ingredient, calculate_margin_cached

N = 2000

def test_mutable_before() -> None:
    d = make_dish_dict(1, "Soup", 100.0)
    for i in range(N):
        add_ingredient_inplace(d, f"ing_{i}", 1.0, 5.0)
    total = sum(calculate_margin_dict(100.0, 50.0) for _ in range(N))
    assert total > 0

def test_immutable_after() -> None:
    d = Dish(1, "Soup", 100.0)
    for i in range(N):
        d = add_ingredient(d, f"ing_{i}", 1.0, 5.0)
    calculate_margin_cached.cache_clear()
    total = sum(calculate_margin_cached(100.0, 50.0) for _ in range(N))
    assert total > 0
    assert calculate_margin_cached.cache_info().hits == N - 1
```

7. Виконав запуск тестів і лінтерів: усі тести проходять успішно, mypy, ruff та black не мають зауважень:

```
PS C:\functional-programming\lab03> python main.py
d0 is d3: False | d0 unchanged: True
Margin: 59.09%
Cache info: CacheInfo(hits=0, misses=1, maxsize=1024, currsize=1)
Indexed 2 dishes successfully as dict keys.

PS C:\functional-programming\lab03> python -m pytest -q --durations=5
............                                                             [100%]
============================== slowest 5 durations ==============================
0.21s call     tests/test_performance.py::test_mutable_before
0.01s call     tests/test_performance.py::test_immutable_after
0.00s call     tests/test_immutability.py::test_add_ingredient_returns_new_dish
12 passed in 0.45s

PS C:\functional-programming\lab03> python -m mypy .
Success: no issues found in 6 source files

PS C:\functional-programming\lab03> python -m ruff check .
All checks passed!

PS C:\functional-programming\lab03> python -m black --check .
All done! 6 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи №3 я перевів мутабельну модель ресторанного меню (варіант 12) на незмінну: сутності Ingredient та Dish реалізовано як frozen dataclass зі slots, інгредієнти у tuple, алергени у frozenset, метадані страви у MappingProxyType над копією джерела. Усі оновлення реалізовано як чисті функції через dataclasses.replace; хешованість дозволила кешувати розрахунок маржі через lru_cache та використовувати страву як ключ словника. Тести підтвердили відсутність мутацій і read-only поведінку, а порівняння часу показало ефективність незмінних підходів завдяки відсутності зайвих захисних копій та мемоізації.